Gorilla: API documents help.
The call still needs checking.

A paper analysis + my Kubernetes experiment | **Gorilla • NeurIPS 2024**

In my own words

Retrieving the right documentation is useful, but it does not guarantee a correct API call—or a refusal when no suitable API is offered.

My scope: retrieval + prompting + AST evaluation.
No fine-tuning.

Paper pp. 1, 5; local notebook and saved experiment results.

Why this matters to my work

01

Understand the request

Map an operations request to one specific Kubernetes API.

02

Use the right version

Ground the answer in the API contract that the cluster serves.

03

Check before action

Verify the function and arguments before considering any execution.

My research question: can documentation guide a verifiable single call?

Paper pp. 1, 5; data/apis.json; data/queries.json; verified answer metadata.

The problem before Gorilla

Too many choices

Many APIs offer similar functionality. A fluent answer can still select the wrong tool.

Changing documents

Versions and arguments evolve. A model's stored knowledge can become stale.

Plausible mistakes

Valid-looking code can contain an invented function or incorrect arguments.

The goal is a usable API call, not merely convincing text.

Paper pp. 1–3 (Introduction and Related Work).

How Gorilla works

TRAINING — paper only; not reproduced in my pilot

API model cards
Three model hubs

→

Synthetic pairs
GPT-4 self-instruct

→

Fine-tune LLaMA-7B
Include API documents

→

Gorilla
Trained model

INFERENCE — the pattern adapted by my pilot

API document store
↓

User request → **Retriever** → **Request + doc** → **Gorilla** → **API call**

Paper: usually top-1 retrieval. **My pilot:** TF-IDF top-3 + Gemma or Qwen.

Redrawn from paper Fig. 3, p. 4; method and retriever-aware training, p. 5.

The key rule: inspect the call's structure

Parse the code → **Match the reference** → **Classify the outcome**

Build an AST.

Function + selected arguments.

Correct, wrong, or invented.

Paper rule

Match API identity; optional arguments need not all match.

My stricter adaptation

One permitted SDK call; verified arguments; no code execution.

Real but absent from the retrieved cards ≠ an invented API.

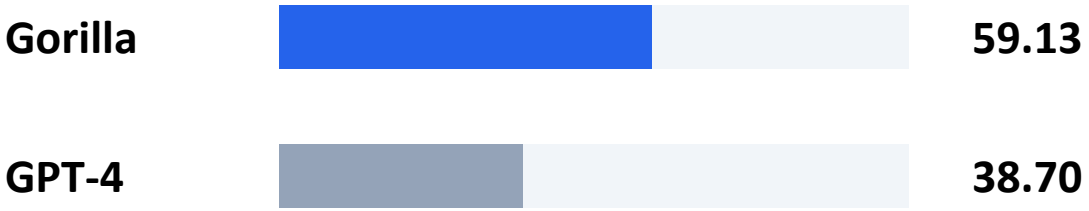
Paper §3.3, pp. 5–6, Fig. 4; notebooks/gorilla_eval.py; refusal registry/evaluator.

Main evidence: a strong TorchHub result

No retriever at inference • fine-tuned Gorilla versus GPT-4

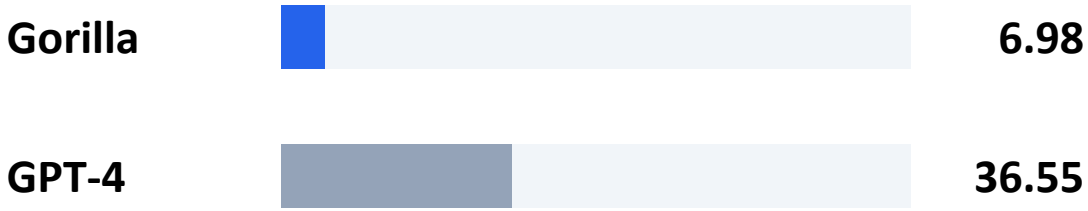
Correct API (%)

Higher is better



Hallucination (%)

Lower is better



Accuracy gain on this setting: +20.43 percentage points.

The comparison changes with the dataset, retrieval method, and scoring rule.

Paper Table 1, p. 8; metric caveats in §4.1, p. 7.

Ablation: training changes how retrieval helps

TorchHub accuracy (%) • rows: training; columns: retrieval at inference

Training documents	None	BM25	GPT-Index	Oracle
None	59.13	37.63	60.21	54.83
Oracle documents	0.00	40.32	61.82	67.20

Oracle → oracle: +12.37 pp

67.20 − 54.83, with the test retriever fixed.

Important counterexample

Oracle-trained model + no retrieved document: 0%.

Paper Table 2, p. 8; retriever-aware training, p. 5; discussion, p. 7.

My reproduction setup

1 Corpus collection

Selected 30 operations / 10 resources from Kubernetes v1.35.2 release and live OpenAPI.

Retained descriptions, arguments, source URLs and versions.

2 Queries and references

20 user-drafted requests, locally revised.

Read/dry-run verified references; 19 distinct correct APIs.

Answers excluded from model input.

3 Models and settings

Gemma-4-31b-it: temp 0; max 4,096 output tokens.

Qwen3-8B (vLLM): temp 0, thinking off; max 3,310 baseline / 3,287 removal.

4 Retrieval and limits

TF-IDF descriptions, cosine, top-3.

20 baseline queries; 10 fixed removals.

Each removal: fresh 29-API index.

One attempt; no code execution.

data/apis.json metadata; query/answer metadata; replacement audit; saved model protocols.

My actual results—and two exact failures

Retrieval: top-1 50% (10/20) • top-3 85% (17/20)

Model	Correct calls	API errors	Refusals: 10 removals	API errors
Gemma (Google Studio)	15% (3/20)	16/20	10% (1/10)	9/10
Qwen (Self-host)	20% (4/20)	0/20	30% (3/10)	0/10

Among successful requests — calls: Gemma 75%, Qwen 20%; refusals: 100% (1/1), 30% (3/10).

API errors are never refusals. Gemma’s one successful removal response is too little to compare.

Two exact invented / unsupported SDK calls (Qwen)

BASELINE · req_005

```
list_resource_v1_device_class(resource_version="0")
```

REMOVAL TEST · req_019

```
scheduling_v1.read_namespaced_priority_class(name="system-cluster-critical", namespace="")
```

refusal/summary.json; qwen_baseline_trace.json req_005; qwen_generation.json req_019.

What I reproduced—and what I did not

Dimension	Paper	My pilot
Training	LLaMA-7B fine-tuning / RAT	None; prompted Gemma and Qwen
Corpus	1,645 model-hub APIs reported	30 selected Kubernetes operations
Retrieval	BM25 / GPT-Index, top-1; oracle	TF-IDF, top-3; ten API removals
Scoring	AST identity; selected arguments	Literal SDK call + verified arguments

Limits: small selected sample; high Gemma API-error rate; no fine-tuning, oracle, or no-retrieval control.

AST checks and prior dry-runs do not prove live execution, device allocation, or workload completion.

Paper pp. 4–8, Table 2; notebook; saved protocols and refusal plan.

Three questions I would ask the authors

01 When should the model refuse?

How should it distinguish a retrieval miss from an API that is genuinely unavailable?

02 How far does the benefit generalize?

Does the gain persist on API-disjoint and time-disjoint tests with retrieval quality controlled?

03 What must be checked beyond the AST?

What extra evidence is needed before a generated call can safely change system state?

Motivated by paper pp. 5, 7–9; Table 2, p. 8; execution discussion, p. 8.

When I would not use this approach alone

Do not use it alone for autonomous writes, stale/missing API docs, or workflows needing execution guarantees.

Before paying customers

Proposed target, not a paper result: ≥99.9% correct refusal on unsupported requests, with a 95% lower confidence bound ≥99.9%.

10/10 would still be weak evidence. About 2,995/2,995 representative independent successes are needed.

Additional evidence and controls

Supported-call accuracy + false refusals.

Held-out APIs, version changes, and real-but-unavailable tools.

API errors, latency, cost, and load tests.

Sandbox execution, allowlists, schema checks, and human review for writes.

My removal results; paper pp. 5–8; NIST exact binomial method (URL in notes).